Data Structures & Algorithms

I selected the leaderboard sorting algorithm as the best function to write manually.

This function sorts my leaderboard by time so the fastest player is shown at the top.

It reads each time entry in the csv file and compares the list of times against each time value.

It looks for the smallest time in the list and when it finds a faster time, it swaps it into the correct position to take first place at the top of the leaderboard or the position that it should be in the list.

Every time the game is played it repeats the process until the whole list is sorted from fastest to slowest.

I used this as I wanted the leaderboard manually sorted instead of using a built-in sort function. I also only wanted to store the top 10 fastest players.

The function works by:-

- using a list to store all of the leaderboard entries.
- adding the new entries to the list as a new record.
- storing the leaderboard as a record (tuple) containing:
    1. the players name
    2. the players game time
    3. the players finishing score

This allows related data to be grouped together in a single structure.

The leaderboard is also stored in a CSV file (leaderboard.csv). This provides permanent storage so data is not lost when the program closes and allows scores to be reloaded later.

See screenshot below-

```python
# Function to manually sort the scores by fastest time
def manual_sort_by_time(scores):  1 usage
    # Manually sorts the leaderboard by the fastest time (lowest first)

    # Gets the number of scores in the list
    n = len(scores)

    # i = the current index where the fastest score will be placed (placing)
    # j = loops through the remaining items to compare times and find a faster score (checking)

    # Loops through each position in the list
    for i in range(n):

        # Assumes the current position (i) is the fastest
        fastest_index = i

        # Looks through the rest of the list (after i)
        for j in range(i + 1, n):

            # Compares the times (index 1 in the tuple)
            # If a smaller time is found, it's faster
            if scores[j][1] < scores[fastest_index][1]:
                fastest_index = j  # Updates the fastest position

        # After checking all the list items, swaps the fastest with position i

        # Stores the current score temporarily
        temp = scores[i]

        # Moves the fastest score into the correct position (position i)
        scores[i] = scores[fastest_index]

        # Puts the original score (stored in temp) into the position where the fastest score was
        scores[fastest_index] = temp

    # Returns the sorted score list
    return scores
```

Other Algorithms

This table shows all the main algorithms used in my project. These include checking answers, reading and saving files, using loops and conditions, and controlling movement, collision checking and timing in the game. Some are based on an event driven code.

Algorithm	Where it is used	What it does
Iteration (loops)	Whole program	Loops through lists like questions, scores, CSV rows, and buttons
Linear search	Questions system	Checks each answer until it finds the correct one
Conditional logic (IF / ELSE)	Whole program	Decides outcomes like correct/wrong answers and game states
Countdown algorithm	Timer system	Reduces time by one second until it reaches 0

Collision detection	Duck + cracker	Compares positions of the objects to see if the duck touches the cracker
Randomisation	Questions system	Shuffles the questions and answer choices randomly to ensure the answer buttons don't end up with the same index
File reading algorithm	CSV loading	Reads and loads the leaderboard and the question data from the csv files line by line
File writing/appending	Leaderboard saving	Adds new names, times and scores into the csv file and saves it
Event-driven handling	Buttons/sounds/UI	Runs code when buttons are clicked or on collision (Tkinter events)
Animation loop algorithm	Duck movement	Repeats movement using after() to create smooth animation